

# **Compiler for an Arithmetic Expression Language using Lex/Flex and Yacc/Bison**

Project Report - Review 1

Metlapalli Mehereswar (23BAI0138) | Team members: <add names / reg. nos.>

B.Tech CSE, VIT Vellore | Compiler Design | Faculty: <add name>

## Abstract

This project builds a compiler for a small arithmetic expression language using Flex and Bison. The full pipeline is: tokenise the source, construct the parse tree, perform semantic validation, and generate Three Address Code (TAC). This Review-1 report covers the first milestone, about 25% of the work: the language definition, the Flex lexer that produces the token stream, and the Bison parser that resolves operator precedence and associativity, recovers from syntax errors, and constructs the parse tree. Semantic validation and TAC generation are the planned next stages.

## 1. Problem Identification

### 1.1 Problem statement

Evaluating or translating an arithmetic expression such as  $a + b * 2 - 4 / 2$  looks trivial, but doing it correctly requires the classical compiler phases: recognising lexemes, respecting operator precedence and associativity, rejecting ill-formed or ill-typed programs, and producing a machine-independent intermediate form. Hand-written ad-hoc evaluators typically fail in four ways:

- Precedence and associativity are encoded in scattered if/else logic and are easy to get wrong (for example,  $2^3^2$  must be  $2^9 = 512$ , not  $8^2 = 64$ ).
- Errors are reported one at a time, or not at all - an undeclared variable or a division by a constant zero surfaces only at run time.
- Mixed int/float arithmetic silently changes meaning unless conversions are made explicit.
- There is no intermediate representation, so no optimisation or retargeting is possible.

### 1.2 Objectives

O1 Specify a small, unambiguous arithmetic language: int/float declarations, assignment, print, operators  $+$   $-$   $*$   $/$   $\%$   $^$ , unary minus, parentheses. [done]

O2 Lexical analysis: generate a token stream with Flex, with line numbers and error reporting for illegal characters. [done]

O3 Syntax analysis: write an LALR(1) grammar in Bison, resolve ambiguity through precedence declarations, and construct the parse tree. [done]

O4 Semantic validation: symbol table, type inference/checking and compile-time diagnostics. [planned]

O5 Intermediate code: generate Three Address Code with temporaries, explicit type conversions and constant folding. [planned]

### 1.3 Scope

In scope: one global scope, two types (int, float), the operators listed above. Out of scope: control flow, functions, arrays, strings, target-machine code generation and register allocation.

## 2. Literature Survey

The design rests on four bodies of work: the phase structure of compilers, lexer/parser generator theory, resolution of expression-grammar ambiguity, and intermediate representations.

| Ref  | Authors                                | Work                                                                                      | Relevance to this project                                                                                               |
|------|----------------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| [1]  | Aho, Lam, Sethi, Ullman                | Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2006                      | Standard reference for the phase structure, syntax-directed translation, three-address code and type checking.          |
| [2]  | M. E. Lesk, E. Schmidt                 | Lex - A Lexical Analyzer Generator, Bell Labs CSTR 39, 1975                               | Original Lex: regular expressions compiled into a DFA-driven scanner. Basis of Flex.                                    |
| [3]  | S. C. Johnson                          | Yacc: Yet Another Compiler-Compiler, Bell Labs CSTR 32, 1975                              | Original Yacc: LALR(1) parser generator with semantic actions. Basis of Bison.                                          |
| [4]  | J. Levine                              | flex & bison, O'Reilly, 2009                                                              | Practical Flex/Bison integration (yyval, %union, token sharing, error recovery).                                        |
| [5]  | R. W. Floyd                            | Syntactic analysis and operator precedence, JACM 10(3), 1963                              | Origin of operator-precedence parsing; the idea behind %left / %right declarations.                                     |
| [6]  | D. E. Knuth                            | On the translation of languages from left to right, Information and Control 8(6), 1965    | Introduced LR(k) parsing, the theory under every Yacc-style generator.                                                  |
| [7]  | A. V. Aho, S. C. Johnson               | LR Parsing, ACM Computing Surveys 6(2), 1974                                              | Survey of LR/LALR construction and conflict handling.                                                                   |
| [8]  | A. V. Aho, S. C. Johnson, J. D. Ullman | Deterministic parsing of ambiguous grammars, CACM 18(8), 1975                             | Resolving shift/reduce conflicts with precedence and associativity - how our ambiguous expr rule is made deterministic. |
| [9]  | K. D. Cooper, L. Torczon               | Engineering a Compiler, 2nd ed., Morgan Kaufmann, 2011                                    | IR design, constant folding and type conversion (planned phases).                                                       |
| [10] | A. W. Appel                            | Modern Compiler Implementation in C, Cambridge Univ. Press, 1998                          | Abstract syntax trees and symbol-table design in C.                                                                     |
| [11] | T. J. Parr, R. W. Quong                | ANTLR: A predicated-LL(k) parser generator, Software: Practice and Experience 25(7), 1995 | Alternative top-down generator; compared with LALR(1) Bison.                                                            |
| [12] | C. Lattner, V. Adve                    | LLVM: A compilation framework for lifelong program analysis & transformation, CGO 2004    | Modern three-address-style SSA IR; shows where TAC leads.                                                               |

### 2.1 Analysis and gap

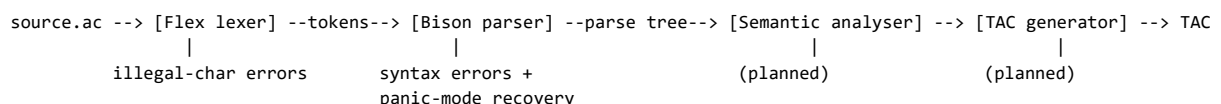
Lex [2] and Yacc [3] established the generator approach that Flex and Bison still follow: regular expressions become a DFA scanner and a context-free grammar becomes an LALR(1) table. Floyd [5] and Aho-Johnson-Ullman [8] show why a deliberately ambiguous rule such as `expr -> expr op expr` is acceptable: precedence and associativity declarations turn shift/reduce conflicts into deterministic choices, giving a smaller grammar than the stratified `expr/term/factor` form. Top-down generators such as ANTLR [11] avoid left-recursion restrictions differently, while LLVM [12] shows the end point of the IR line that TAC begins.

Textbook treatments [1][9][10] present the phases separately and usually stop at the parse tree or at TAC listings. This project aims at a small, integrated pipeline in which every phase can be run and observed on the same input; Review 1 delivers the front half of it.

Note: citations should be re-checked against the library copies before final submission.

## 3. Proposed System and Design

### 3.1 Architecture



|<----- Review 1: implemented ----->|

## 3.2 Language grammar

```
program -> stmts
stmts   -> stmts stmt | (empty)
stmt    -> type ID ';'
        | type ID '=' expr ';'
        | ID '=' expr ';'
        | 'print' expr ';'
        | error ';'                (panic-mode recovery)
type    -> 'int' | 'float'
expr    -> expr ('+' | '-' | '*' | '/' | '%' | '^') expr
        | '-' expr | '+' expr | '(' expr ')'
```

Precedence (low -> high):    + -    <    \* / %    <    unary -    <    ^  
Associativity:                    left for + - \* / %,    right for ^

## 3.3 Lexical specification (Flex)

| Token class                            | Pattern / examples               | Purpose                           |
|----------------------------------------|----------------------------------|-----------------------------------|
| KEYWORD                                | int float print                  | Reserved words                    |
| IDENTIFIER                             | [A-Za-z][A-Za-z0-9_]*            | Variable names                    |
| INT_LITERAL                            | [0-9]+                           | Integer constants                 |
| FLOAT_LITERAL                          | 1.5 .5 2. 1e3 2.5E-2             | Real constants, optional exponent |
| PLUS MINUS STAR SLASH<br>PERCENT CARET | + - * / % ^                      | Arithmetic operators              |
| ASSIGN                                 | =                                | Assignment                        |
| LPAREN RPAREN SEMICOLON                | ( ) ;                            | Punctuation                       |
| (skipped)                              | whitespace, // line, /* block */ | Discarded by the lexer            |

Flex uses the longest-match rule, and keyword rules are listed before the identifier rule so that 'int' is a KEYWORD and not an IDENTIFIER. %option yylineno supplies line numbers. A character that matches no rule is reported as a lexical error and scanning continues.

## 3.4 Parser and parse-tree construction (Bison)

Each grammar action calls a node constructor (mk\_bin, mk\_int, mk\_decl ...) so that a syntax tree is assembled bottom-up during parsing. The %union carries integer, float, string and node values. The rule 'error ;' implements panic-mode recovery: after a syntax error the parser discards tokens up to the next semicolon and continues, so one run reports all errors in the file.

Ambiguity is resolved with precedence declarations (%left '+' '-', then %left '\*' '/' '%', %precedence UMINUS, %right '^'). Making '^' bind tighter than unary minus follows mathematical convention (-2^2 = -4). Bison builds the grammar with no warnings and no conflicts.

## 3.5 Planned phases (next reviews)

| Phase               | Planned design                                                                                                                                                                              |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Semantic validation | Symbol table (name, type, initialised); type inference with int-to-float promotion; errors for undeclared variable, redeclaration, % on floats, constant division by zero; warnings for use |

|                    |                                                                                                                                                                            |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    | before assignment and float-to-int narrowing.                                                                                                                              |
| Three Address Code | At most one operator per instruction; temporaries <code>_t1, _t2 ...</code> ; explicit <code>inttofloat / floattoint</code> ; constant folding of literal-only operations. |

## 4. Implementation Status

Implementation is in C with Flex 2.6.4, Bison 3.8.2 and GCC. The table maps each Review-1 module to its source file.

| Module                                 | File                               | State                 |
|----------------------------------------|------------------------------------|-----------------------|
| Lexical analyser (token stream)        | src/lexer.l, src/ast.c (token log) | Done - tested         |
| Parser (grammar, precedence, recovery) | src/parser.y                       | Done - tested         |
| Parse-tree construction and printer    | src/ast.c                          | Done - tested         |
| Driver                                 | src/main.c                         | Done                  |
| Regression suite                       | run_tests.sh                       | 12 / 12 passing       |
| Semantic analysis                      | -                                  | Not started (planned) |
| Three Address Code                     | -                                  | Not started (planned) |

### 4.1 Build and run

```
sh build.sh                # bison -d, flex, gcc -> arithc.exe
./arithc.exe prog.ac        # diagnostics, token stream, parse tree
./arithc.exe -q prog.ac     # quiet: hide the token table
sh run_tests.sh             # regression suite
```

## 5. Results

### 5.1 Token stream

Source (tests/t4\_demo.ac):

```
int a = 10;
int b = 3;
float x = 2.5;
int c = a + b * 2 - 4 / 2;
float y = x * (a - b);
print c;
print y;
```

Output of the lexer (first 26 lines):

| LINE | TOKEN       | LEXEME |
|------|-------------|--------|
| 1    | KEYWORD     | int    |
| 1    | IDENTIFIER  | a      |
| 1    | ASSIGN      | =      |
| 1    | INT_LITERAL | 10     |
| 1    | SEMICOLON   | ;      |
| 2    | KEYWORD     | int    |
| 2    | IDENTIFIER  | b      |
| 2    | ASSIGN      | =      |

```

2    INT_LITERAL    3
2    SEMICOLON      ;
3    KEYWORD        float
3    IDENTIFIER      x
3    ASSIGN          =
3    FLOAT_LITERAL  2.5
3    SEMICOLON      ;
4    KEYWORD        int
4    IDENTIFIER      c
4    ASSIGN          =
4    IDENTIFIER      a
4    PLUS            +
4    IDENTIFIER      b
4    STAR            *
4    INT_LITERAL     2
4    MINUS           -
4    INT_LITERAL     4

```

## 5.2 Parse tree

Parse tree for the same program. '\*' binds tighter than '+', and '-' is left-associative, so  $\text{int } c = a + b * 2 - 4 / 2$  parses as  $(a + (b * 2)) - (4 / 2)$ :

```

Program
|-- Decl int a
|  |-- Int 10
|-- Decl int b
|  |-- Int 3
|-- Decl float x
|  |-- Float 2.5
|-- Decl int c
|  |-- BinOp '-'
|     |-- BinOp '+'
|     |  |-- Var a
|     |  |-- BinOp '*'
|     |     |-- Var b
|     |     |-- Int 2
|     |-- BinOp '/'
|     |  |-- Int 4
|     |-- Int 2
|-- Decl float y
|  |-- BinOp '*'
|     |-- Var x
|     |-- BinOp '-'
|     |  |-- Var a
|     |  |-- Var b
|-- Print
|  |-- Var c
|-- Print
|  |-- Var y

```

## 5.3 Associativity of ^

For  $\text{int } p = 2 \wedge 3 \wedge 2$ ; the second ^ is the right child of the first, i.e.  $2 \wedge (3 \wedge 2) = 512$ , as required:

```

|-- Decl int p
|  |-- BinOp '^'

```

```

|           |-- Int 2
|           `-- BinOp '^'
|           |-- Int 3
|           `-- Int 2
|-- Decl int r

```

## 5.4 Lexical and syntax error recovery

Source (tests/t3\_syntax.ac):

```

int a = 4;
int b = (a + ;
int c = a $ 2;
print a +* 2;
int d = a * 2;
print d;

```

Diagnostics: every bad line is reported and parsing continues to the end of the file.

```

== Phase 1+2: lexing and parsing (diagnostics) ==
line 2: error: syntax error, unexpected ';'
line 3: error: lexical error: illegal character '$'
line 3: error: syntax error, unexpected INUM
line 4: error: syntax error, unexpected '*'

```

## 5.5 Regression suite

```

PASS keyword token
PASS float literal token
PASS caret operator token
PASS comments are skipped
PASS * binds tighter than +
PASS left-assoc minus
PASS unary minus node
PASS valid file: 0 errors
PASS illegal character reported
PASS syntax error line 2
PASS recovery reaches line 4
PASS ^ nests as 2^(3^2)

```

## 6. Project Plan

| Phase | Work                                                                                                  | Status                 |
|-------|-------------------------------------------------------------------------------------------------------|------------------------|
| 1     | Language definition, grammar, token set                                                               | Done - Review 1        |
| 2     | Flex lexer: token stream, line numbers, lexical errors                                                | Done - Review 1        |
| 3     | Bison parser: precedence, parse-tree construction, error recovery                                     | Done - Review 1        |
| 4     | Symbol table and semantic validation (declarations, type checking, int/float rules, division by zero) | Planned - next review  |
| 5     | Three Address Code generation (temporaries, type conversions, constant folding)                       | Planned - next review  |
| 6     | Testing of semantic and code-generation phases; final report and demo                                 | Planned - final review |

## 7. Conclusion

Review 1 delivers the language definition, a Flex lexer producing a line-numbered token stream, and a Bison parser that builds the parse tree, resolves precedence and associativity through declarations, and recovers from syntax errors so that all errors are reported in one run. The remaining phases - semantic validation and Three Address Code generation - build directly on the parse tree produced here.

## References

- [1] Aho, Lam, Sethi, Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2006.
- [2] M. E. Lesk, E. Schmidt, Lex - A Lexical Analyzer Generator, Bell Labs CSTR 39, 1975.
- [3] S. C. Johnson, Yacc: Yet Another Compiler-Compiler, Bell Labs CSTR 32, 1975.
- [4] J. Levine, flex & bison, O'Reilly, 2009.
- [5] R. W. Floyd, Syntactic analysis and operator precedence, JACM 10(3), 1963.
- [6] D. E. Knuth, On the translation of languages from left to right, Information and Control 8(6), 1965.
- [7] A. V. Aho, S. C. Johnson, LR Parsing, ACM Computing Surveys 6(2), 1974.
- [8] A. V. Aho, S. C. Johnson, J. D. Ullman, Deterministic parsing of ambiguous grammars, CACM 18(8), 1975.
- [9] K. D. Cooper, L. Torczon, Engineering a Compiler, 2nd ed., Morgan Kaufmann, 2011.
- [10] A. W. Appel, Modern Compiler Implementation in C, Cambridge Univ. Press, 1998.
- [11] T. J. Parr, R. W. Quong, ANTLR: A predicated-LL(k) parser generator, Software: Practice and Experience 25(7), 1995.
- [12] C. Lattner, V. Adve, LLVM: A compilation framework for lifelong program analysis & transformation, CGO 2004.

## Appendix A - Source listing

### src/lexer.l

```
%{
/* lexer.l - Phase 1: lexical analysis */
#include <stdlib.h>
#include <string.h>
#include "ast.h"
#include "parser.tab.h"

static int tok(const char *cls, int code) {
```

```

        tok_record(cls, yytext, yylineno);
        return code;
    }
}

%option noyywrap yylineno nounput noinput

DIGIT    [0-9]
ID        [A-Za-z][A-Za-z0-9_]*
EXP       [eE][+-]?{DIGIT}+

%x BLOCK

%%
[ \t\r]+      ;
\n            ;
"//".*        ;
"/*"          { BEGIN(BLOCK); }
<BLOCK>"*/"    { BEGIN(INITIAL); }
<BLOCK>.|\\n    ;
<BLOCK><<EOF>>  { report("error", yylineno, "unterminated comment"); BEGIN(INITIAL); }

"int"         { return tok("KEYWORD", KW_INT); }
"float"       { return tok("KEYWORD", KW_FLOAT); }
"print"       { return tok("KEYWORD", KW_PRINT); }

{DIGIT}+"."{DIGIT}*{EXP}? |
"."{DIGIT}+{EXP}?         |
{DIGIT}+{EXP}             { yylval.fval = atof(yytext); return tok("FLOAT_LITERAL", FNUM); }
{DIGIT}+                 { yylval.ival = atoll(yytext); return tok("INT_LITERAL", INUM); }

{ID}              { yylval.str = strdup(yytext); return tok("IDENTIFIER", ID); }

"+"             { return tok("PLUS", '+'); }
"_"             { return tok("MINUS", '-'); }
"*"             { return tok("STAR", '*'); }
"/"            { return tok("SLASH", '/'); }
"%"            { return tok("PERCENT", '%'); }
"^"            { return tok("CARET", '^'); }
"="            { return tok("ASSIGN", '='); }
"("            { return tok("LPAREN", '('); }
")"            { return tok("RPAREN", ')'); }
";"            { return tok("SEMICOLON", ';'); }

.              { report("error", yylineno, "lexical error: illegal character '%s'", yytext); }
%%

```

## src/parser.y

```

%{
/* parser.y - Phase 2: syntax analysis, builds the syntax (parse) tree */
#include <stdio.h>
#include "ast.h"
int yylex(void);
void yyerror(const char *msg);
extern int yylineno;
%}

#define parse.error verbose

```

```

%union {
    long long ival;
    double fval;
    char *str;
    struct Node *node;
}

%token KW_INT KW_FLOAT KW_PRINT
%token <ival> INUM
%token <fval> FNUM
%token <str> ID
%type <node> expr stmt
%type <ival> type

/* lowest to highest precedence; '^' is right-assoc and binds tighter than unary minus */
%left '+' '-'
%left '*' '/' '%'
%precedence UMINUS
%right '^'

%%

program : stmts ;

stmts   : %empty
        | stmts stmt      { if ($2) add_stmt($2); }
        ;

stmt    : type ID ';'      { $$ = mk_decl((Type)$1, $2, NULL, yylineno); }
        | type ID '=' expr ';' { $$ = mk_decl((Type)$1, $2, $4, yylineno); }
        | ID '=' expr ';'   { $$ = mk_assign($1, $3, yylineno); }
        | KW_PRINT expr ';' { $$ = mk_print($2, yylineno); }
        | error ';'        { $$ = NULL; yyerrok; } /* panic-mode recovery */
        ;

type    : KW_INT           { $$ = TY_INT; }
        | KW_FLOAT        { $$ = TY_FLOAT; }
        ;

expr    : expr '+' expr    { $$ = mk_bin('+', $1, $3, yylineno); }
        | expr '-' expr    { $$ = mk_bin('-', $1, $3, yylineno); }
        | expr '*' expr    { $$ = mk_bin('*', $1, $3, yylineno); }
        | expr '/' expr    { $$ = mk_bin('/', $1, $3, yylineno); }
        | expr '%' expr    { $$ = mk_bin('%', $1, $3, yylineno); }
        | expr '^' expr    { $$ = mk_bin('^', $1, $3, yylineno); }
        | '-' expr %prec UMINUS { $$ = mk_neg($2, yylineno); }
        | '+' expr %prec UMINUS { $$ = $2; }
        | '(' expr ')'      { $$ = $2; }
        | INUM              { $$ = mk_int($1, yylineno); }
        | FNUM              { $$ = mk_float($1, yylineno); }
        | ID                { $$ = mk_var($1, yylineno); }
        ;

%%

void yyerror(const char *msg) {
    report("error", yylineno, "%s", msg);
}

```

## src/ast.h

```

/* ast.h - shared definitions: token log, syntax tree, diagnostics */
#ifndef AST_H
#define AST_H

```

```

#include <stdio.h>

typedef enum { TY_INT, TY_FLOAT } Type;
typedef enum { N_DECL, N_ASSIGN, N_PRINT, N_BIN, N_NEG, N_INT, N_FLOAT, N_VAR } Kind;

typedef struct Node {
    Kind kind;
    int line;
    char op;          /* '+', '-', '*', '/', '%', '^' for N_BIN */
    char *name;       /* variable name (DECL/ASSIGN/VAR) */
    long long ival;   /* N_INT */
    double fval;      /* N_FLOAT */
    Type type;        /* declared type (N_DECL) */
    struct Node *l, *r; /* children; DECL/ASSIGN use l for init expr, PRINT uses l */
    struct Node *next; /* statement list */
} Node;

extern int n_errors;
void report(const char *kind, int line, const char *fmt, ...);

/* token log (filled by the lexer) */
void tok_record(const char *cls, const char *lexeme, int line);
void tok_dump(FILE *f);

/* tree construction (called from Bison actions) */
Node *mk_int(long long v, int line);
Node *mk_float(double v, int line);
Node *mk_var(char *name, int line);
Node *mk_bin(char op, Node *l, Node *r, int line);
Node *mk_neg(Node *e, int line);
Node *mk_decl(Type t, char *name, Node *init, int line);
Node *mk_assign(char *name, Node *e, int line);
Node *mk_print(Node *e, int line);
void add_stmt(Node *s);
Node *program(void);

void tree_dump(FILE *f, Node *prog);
const char *type_name(Type t);
#endif

```

## src/main.c

```

/* main.c - Review-1 driver: lexer + parser + parse tree */
#include <stdio.h>
#include <string.h>
#include "ast.h"

extern FILE *yyin;
int yyparse(void);

int main(int argc, char **argv) {
    int quiet = 0; const char *path = NULL;
    for (int i = 1; i < argc; i++) {
        if (!strcmp(argv[i], "-q")) quiet = 1;    /* hide the token table */
        else path = argv[i];
    }
    if (path && !(yyin = fopen(path, "r"))) { perror(path); return 2; }

    printf("== Phase 1+2: lexing and parsing (diagnostics) ==\n");
    yyparse();
    if (n_errors == 0) printf(" none\n");
}

```

```
if (!quiet) { printf("\n== Token stream ==\n"); tok_dump(stdout); }

printf("\n== Syntax (parse) tree ==\n");
tree_dump(stdout, program());

printf("\n== Summary: %d error(s) ==\n", n_errors);
return n_errors != 0;
}
```